Formalizing Gödel's Incompleteness in a Binary Tree Calculus: A Guard/Rose-Style Approach in Agda

March 2026

Abstract

We describe an Agda formalization of Gödel's incompleteness theorems in a binary tree calculus inspired by Rose's system B* and Guard's 1963 lecture notes. The formalization uses intrinsically scoped de Bruijn terms over binary trees, with primitive recursion (rec) and right-spine iteration (niter) as the computational primitives.

The main results include: (1) a coded diagonal operator with a correctness proof linking tree-level and syntactic substitution; (2) an internal theorem enumerator thS with reflexivity, symmetry, transitivity, and computation axioms (cas-leaf, rec-leaf), together with a full correctness proof; (3) a semantic soundness theorem via a validity predicate on proof trees (ValidProof) and a decoded-equation witness (ThSResult); (4) a proof-quoting operator D_{thS} with metalevel and internal correctness; (5) a decode round-trip lemma decodeTerm n (codeTerm t) = just t; (6) a Gödel sentence construction with both code-level and semantic fixed-point theorems; (7) Gödel's First Incompleteness Theorem: a specific *true* equation (niter lf lf (pair v_1 v_0) = leaf, both sides evaluating to lf) is proved to lie outside the range of thS via a novel *coreTree quotient* technique; and (8) meta-level consistency and a Rose-style Second Incompleteness Theorem: thS never produces the false equation code, and the self-referential Gödel consistency sentence is true but unprovable.

All proofs use Agda's `--safe` flag with no postulates, no holes, and no `with`-abstraction.

1 The Object System

1.1 Binary Trees

The sole data type is:

$$\mathsf{Tree} ::= \mathsf{lf} \mid \mathsf{nd}(a, b)$$

This is Rose's system B*, where all computation operates on binary trees.

1.2 Terms

Terms are intrinsically scoped using de Bruijn indices:

$$\mathsf{Term}\ n ::= \mathsf{var}\ i \mid \mathsf{leaf} \mid \mathsf{pair}\ t\ u \mid \mathsf{cas}\ t\ u\ v \mid \mathsf{rec}\ t\ z\ s \mid \mathsf{niter}\ t\ st\ s$$

where n is the number of free variables, cas binds 2 variables in v, rec binds 4 variables in s, and niter binds 2 variables in s.

The computation rules are:

$$\begin{aligned}
\mathsf{cas}\ \mathsf{lf}\ u\ v &\to u \\
\mathsf{cas}\ (\mathsf{nd}\ a\ b)\ u\ v &\to v[b/0,\ a/1] \\
\mathsf{rec}\ \mathsf{lf}\ z\ s &\to z \\
\mathsf{rec}\ (\mathsf{nd}\ a\ b)\ z\ s &\to s[\mathsf{rec}\ b\ z\ s/0,\ \mathsf{rec}\ a\ z\ s/1,\ b/2,\ a/3] \\
\mathsf{niter}\ \mathsf{lf}\ st\ s &\to st \\
\mathsf{niter}\ (\mathsf{nd}\ a\ k)\ st\ s &\to \mathsf{niter}\ k\ (s[k/0,\ st/1])\ s
\end{aligned}$$

1.3 Evaluation and Coding

Environment-based evaluation gives $\mathsf{eval} : \mathsf{Term}\ 0 \to \mathsf{Tree}$. Reification injects trees as closed terms: $\mathsf{reify} : \mathsf{Tree} \to \mathsf{Term}\ 0$, with the round-trip property $\mathsf{eval}(\mathsf{reify}\ t) = t$.

Coding maps terms to trees: $\mathsf{codeTerm} : \mathsf{Term}\ n \to \mathsf{Tree}$, using a uniform nd *tag payload* encoding.

2 Coded Substitution and the Diagonal

2.1 Coded Substitution

The function $\mathsf{codedSubst}$ *repl* k *code* performs substitution at the tree (code) level, replacing the k-th variable in the coded term *code* with *repl*.

Theorem 1 (Coded Substitution Correctness)**.** *For all k, $r : \mathsf{Term}\ 0$, $t : \mathsf{Term}\ (n{+}1)$:*

$$\mathsf{codedSubst}\ (\mathsf{codeTerm}\ r)\ k\ (\mathsf{codeTerm}\ t) = \mathsf{codeTerm}\ (\mathsf{substAt}\ k\ r\ t)$$

2.2 The Diagonal Operator

Definition 2. $\mathsf{codeReify} : \mathsf{Tree} \to \mathsf{Tree}$ *computes* $\mathsf{codeTerm}(\mathsf{reify}\ t)$ *directly on trees. The diagonal operator is:*

$$\mathsf{diagCode}\ c = \mathsf{codedSubst}_0\ (\mathsf{codeReify}\ c)\ c$$

This is Guard's $\mathsf{sub}(i, i)$*: substitute the numeral of c into c itself.*

Theorem 3 (Diagonal Correctness)**.**

$$\mathsf{diagCode}\ (\mathsf{codeTerm}\ t) = \mathsf{codeTerm}\ (\mathsf{substAt}_0\ (\mathsf{reify}\ (\mathsf{codeTerm}\ t))\ t)$$

2.3 The Fixed-Point Construction

Definition 4. *For any schema $A : \mathsf{Term}\ 1$, the diagonalization is:*

$$\mathsf{diag}\ A = \mathsf{substAt}_0\ (\mathsf{reify}\ (\mathsf{codeTerm}\ A))\ A$$

Theorem 5 (Fixed-Point Identity)**.**

$$\mathsf{codeTerm}\ (\mathsf{diag}\ A) = \mathsf{diagCode}\ (\mathsf{codeTerm}\ A)$$

3 The Decode Round-Trip

Decoding inverts coding. The function $\mathsf{decodeTerm}\ n : \mathsf{Tree} \to \mathsf{Maybe}\ (\mathsf{Term}\ n)$ recovers a term from its tree code.

Theorem 6 (Decode Round-Trip)**.** *For all* $t : \mathsf{Term}\ n$*:*

$$\mathsf{decodeTerm}\ n\ (\mathsf{codeTerm}\ t) = \mathsf{just}\ t$$

The proof is by mutual structural induction on t, using subsidiary lemmas for natural numbers ($\mathsf{decodeNat}\ (\mathsf{natCode}\ k) = \mathsf{just}\ k$) and finite indices ($\mathsf{natToFin}\ n\ (\mathsf{finToNat}\ i) = \mathsf{just}\ i$).

Corollary 7 (Evaluation of Coded Terms)**.** *Defining* $\mathsf{evalCode}\ c = \mathsf{maybeMap}\ \mathsf{eval}\ (\mathsf{decodeTerm}\ 0\ c)$*, we have:*

$$\mathsf{evalCode}\ (\mathsf{codeTerm}\ t) = \mathsf{just}\ (\mathsf{eval}\ t)$$

Corollary 8 (Code Injectivity on Evaluation)**.** *If* $\mathsf{codeTerm}\ t_1 = \mathsf{codeTerm}\ t_2$ *(as trees), then* $\mathsf{eval}\ t_1 = \mathsf{eval}\ t_2$*.*

This follows from the round-trip via: $\mathsf{just}\ (\mathsf{eval}\ t_1) = \mathsf{evalCode}\ (\mathsf{codeTerm}\ t_1) = \mathsf{evalCode}\ (\mathsf{codeTerm}\ t_2) = \mathsf{just}\ (\mathsf{eval}\ t_2)$, then just-injectivity.

4 The Theorem Enumerator

4.1 Metalevel Definition

The function $\mathsf{thS} : \mathsf{Tree} \to \mathsf{Tree}$ enumerates coded equations (trees of the form $\mathsf{nd}\ L\ R$ representing equations $L = R$). Proof trees encode:

- lf: default equation ($\mathsf{leaf} = \mathsf{leaf}$)
- $\mathsf{nd}\ \mathsf{lf}\ \mathit{payload}$: reflexivity ($\mathsf{codeReify}(\mathit{payload}) = \mathsf{codeReify}(\mathit{payload})$)
- $\mathsf{nd}\ (\mathsf{nd}\ \mathsf{lf}\ \mathsf{lf})\ \mathit{payload}$: symmetry of $\mathsf{thS}(\mathit{payload})$
- $\mathsf{nd}\ (\mathsf{nd}\ \mathsf{lf}\ (\mathsf{nd}\ \mathsf{lf}\ (\mathsf{nd}\ u\ v)))\ \mathit{payload}$: cas-leaf axiom $\mathsf{cas}\ \mathsf{lf}\ U\ V = U$ (where $u = \mathsf{codeTerm}\ U$, $v = \mathsf{codeTerm}\ V$)
- $\mathsf{nd}\ (\mathsf{nd}\ \mathsf{lf}\ (\mathsf{nd}\ (\mathsf{nd}\ \mathsf{lf}\ \mathsf{lf})\ (\mathsf{nd}\ z\ s)))\ \mathit{payload}$: rec-leaf axiom $\mathsf{rec}\ \mathsf{lf}\ Z\ S = Z$ (where $z = \mathsf{codeTerm}\ Z$, $s = \mathsf{codeTerm}\ S$)
- $\mathsf{nd}\ (\mathsf{nd}\ (\mathsf{nd}\ a\ b)\ c)\ \mathit{payload}$: transitivity of $\mathsf{thS}(\mathsf{nd}\ (\mathsf{nd}\ a\ b)\ c)$ and $\mathsf{thS}(\mathit{payload})$, with a tree-equality check on the middle terms

4.2 Internal Representation

Definition 9. $\mathsf{thTerm} : \mathsf{Term}\ 1$ *is a closed term that computes* thS *internally. It uses* rec *for structural recursion on the proof tree, nested* cas *for dispatching on the proof structure (including an* axiom dispatch *sub-term for the cas-leaf and rec-leaf branches), and an* niter*-based work-stack machine for the tree-equality check in the transitivity branch.*

Theorem 10 (Internal Theorem Enumerator Correctness)**.** *For all* $t : \mathsf{Tree}$*:*

$$\mathsf{eval}_{\rho[t/0]}\ \mathsf{thTerm} = \mathsf{thS}\ t$$

5 Internal Tree Equality

5.1 The Challenge

Tree equality requires simultaneous structural recursion on two arguments. The rec primitive gives recursion on *one* argument, and the results for the children compare each child against the *whole* second argument, not against the corresponding child. A direct rec-based approach fails.

5.2 The Solution: Linearize + Niter Work-Stack

We decompose tree equality into three phases:

1. **Linearize**: rec on both trees to produce a right-leaning tree (the "clock") with right-spine length equal to the total node count.
2. **Work-stack machine**: niter on the clock, maintaining a state nd *flag stack* where *stack* is a list of pairs to compare. Each step pops a pair, checks leaf/node agreement, and pushes child pairs for node-node matches.
3. **Extract flag**: a final cas to read the flag (lf for equal, nd lf lf for unequal).

Theorem 11 (Internal Tree Equality Correctness)**.**

$$\mathsf{extractFlagMeta}\ (\mathsf{runNiter}\ (\mathsf{linearize}\ (\mathsf{nd}\ a\ b))\ (\mathsf{nd}\ \mathsf{lf}\ (\mathsf{nd}\ (\mathsf{nd}\ a\ b)\ \mathsf{lf}))) = \mathsf{eqTree}\ a\ b$$

6 Soundness of the Theorem Enumerator

6.1 Unconditional Soundness

Theorem 12 (nd-Headedness)**.** *For all* $y : \mathsf{Tree}$*:*

$$\mathsf{eqTree}\ (\mathsf{thS}\ y)\ \mathsf{lf} = \mathsf{falseT}$$

This says every thS output is of the form nd L R (never lf). The proof handles symmetry via swapCode (which always returns nd-form) and transitivity via transCode (which returns nd-form or defaultCode = nd (nd lf lf) (nd lf lf)).

6.2 The Validity Predicate

The thS function is total on all trees, but only produces semantically meaningful equation codes when the input is a well-formed proof tree. In particular, the computation axiom branches (cas-leaf, rec-leaf) carry term codes as data, which must actually be valid codeTerm images.

Definition 13 (Valid Proof)**.** IsTermCode n c *holds when* $c =$ codeTerm t *for some* t : Term n*.* ValidProof : Tree → Set *is an inductive predicate mirroring the structure of* thS*:*

- *Base (*lf*) and reflexivity (*nd lf *payload): always valid.*
- *Symmetry: valid if the sub-proof is valid.*
- *Cas-leaf: valid if* IsTermCode 0 u *and* IsTermCode 2 v*.*
- *Rec-leaf: valid if* IsTermCode 0 z *and* IsTermCode 4 s*.*
- *Transitivity: valid if both sub-proofs are valid.*
- *Default branches: always valid (produce* defaultCode*).*

6.3 The TrueEqCode Predicate

Definition 14.

$$\mathsf{TrueEqCode}\ c = \begin{cases} \mathsf{Unit} & \textit{if } c = \mathsf{If} \\ \mathsf{trueEqMaybe}\ (\mathsf{evalCode}\ L)\ (\mathsf{evalCode}\ R) & \textit{if } c = \mathsf{nd}\ L\ R \end{cases}$$

where trueEqMaybe (just a) (just b) = Eq a b, *and all other cases give* Unit *(vacuous truth).*

Remark 15 (Vacuous Truth and Transitivity)**.** TrueEqCode *with vacuous truth is preserved by* swapCode *(symmetry) but* not *by* transCode *(transitivity). If one side of an equation code fails to decode while the other succeeds, vacuous truth holds locally, but composing two such codes via* transCode *can yield a goal* Eq a c *that is unprovable. This is the fundamental reason why soundness must be restricted to valid proof trees.*

6.4 The ThSResult Witness

Definition 16. ThSResult c *is inhabited when* c = nd (codeTerm l) (codeTerm r) *for some* l, r : Term 0 *with* eval l = eval r. *This is strictly stronger than* TrueEqCode *because it guarantees both sides decode (no vacuous truth).*

Lemma 17 (Swap Preserves ThSResult)**.** *If* ThSResult c *then* ThSResult (swapCode c)*.*

Lemma 18 (TransCode Preserves ThSResult)**.** *If* ThSResult e_1 *and* ThSResult e_2 *then* ThSResult (transCode e_1 e_2)*.*

The transitivity case works because ThSResult guarantees both sides are codeTerm images. When eqTree matches the middle terms (right of e_1 equals left of e_2 as trees), Corollary 8 gives the evaluation bridge: eval r_1 = eval l_2, which composes with the sub-proofs via eqTrans.

6.5 The Main Soundness Theorem

Theorem 19 (Soundness of thS)**.** *For all* p : Tree*:*

$$\mathsf{ValidProof}\ p\ \rightarrow\ \mathsf{ThSResult}\ (\mathsf{thS}\ p)$$

and consequently:

$$\mathsf{ValidProof}\ p\ \rightarrow\ \mathsf{TrueEqCode}\ (\mathsf{thS}\ p)$$

The proof is by induction on ValidProof p:

- *Reflexivity*: thS produces nd (codeReify a) (codeReify a). By codeReify-correct, this equals nd (codeTerm (reify a)) (codeTerm (reify a)), and eval of both sides is the same (reflexivity).
- *Cas-leaf*: with IsTermCode witnesses u = codeTerm U and v = codeTerm V, the output is nd (codeTerm (cas leaf U V)) (codeTerm U). By the computation rule cas If U $V \rightarrow U$, both sides evaluate to eval U, so the proof is refl.
- *Rec-leaf*: analogous, using rec If Z $S \rightarrow Z$.
- *Symmetry*: by the inductive hypothesis and Swap preservation.
- *Transitivity*: by the inductive hypothesis and Lemma 18.
- *Default branches*: produce defaultCode = nd (codeTerm leaf) (codeTerm leaf), trivially true.

7 The Proof-Quoting Operator D_{thS}

Definition 20. *The metalevel proof-quoting operator is:*

$$D_{\mathsf{thS}}\ p = \mathsf{nd}\ \mathsf{lf}\ (\mathsf{thS}\ p)$$

The internal term is $\mathsf{dThSTerm} = \mathsf{pair}\ \mathsf{leaf}\ \mathsf{thTerm} : \mathsf{Term}\ 1$.

This uses the reflexivity branch of thS: the proof tree nd lf *payload* proves $\mathsf{codeReify}(\mathit{payload}) = \mathsf{codeReify}(\mathit{payload})$. Setting $\mathit{payload} = \mathsf{thS}\ p$, we obtain a proof that $\mathsf{thS}(p) = \mathsf{thS}(p)$.

Theorem 21 (D_thS Correctness — Metalevel)**.**

$$\mathsf{thS}\ (D_{\mathsf{thS}}\ p) = \mathsf{nd}\ (\mathsf{codeReify}\ (\mathsf{thS}\ p))\ (\mathsf{codeReify}\ (\mathsf{thS}\ p))$$

Proof. Definitional (by the reflexivity case of thS). □

Theorem 22 (D_thS Correctness — Internal)**.**

$$\mathsf{eval}_{\rho[p/0]}\ \mathsf{dThSTerm} = D_{\mathsf{thS}}\ p$$

Proof. By eqCong (nd lf) (thTerm-correct p). □

Proposition 23 (D_thS Validity and Soundness)**.**

(i) $\mathsf{ValidProof}\ (D_{\mathsf{thS}}\ p)$ *for all* p.

(ii) $\mathsf{TrueEqCode}\ (\mathsf{thS}\ (D_{\mathsf{thS}}\ p))$ *for all* p.

Proof. (i) By vpRefl (thS p). (ii) By trueEq-refl (thS p). □

8 The Gödel Sentence

Definition 24. *For a target tree target:*

$$\mathsf{godelSchema}\ \mathit{target} = \mathsf{matchSub}\ \mathit{target}\ \mathsf{thTerm}$$
$$\mathsf{godelSentence}\ \mathit{target} = \mathsf{diag}\ (\mathsf{godelSchema}\ \mathit{target})$$

where matchSub *target* t *checks whether* eval t *equals target by structural decomposition.*

Theorem 25 (Gödel Code-Level Fixed Point)**.**

$$\mathsf{codeTerm}\ (\mathsf{godelSentence}\ \mathit{target}) = \mathsf{diagCode}\ (\mathsf{codeTerm}\ (\mathsf{godelSchema}\ \mathit{target}))$$

Theorem 26 (Gödel Semantic Fixed Point)**.**

$$\mathsf{eval}\ (\mathsf{godelSentence}\ \mathit{target}) = \mathsf{eqTree}\ (\mathsf{thS}\ (\mathsf{codeTerm}\ (\mathsf{godelSchema}\ \mathit{target})))\ \mathit{target}$$

Theorem 27 (Gödel Sentence Evaluation)**.** *For target* = lf*:*

$$\mathsf{eval}\ (\mathsf{godelSentence}\ \mathsf{lf}) = \mathsf{falseT}$$

Proof. By the semantic fixed point and Theorem 12: thS always returns an nd-headed tree, so eqTree (thS c) lf = falseT. □

9 The CoreTree Quotient and Gödel I

9.1 The CoreTree Function

The key technical innovation is a tree-level function $\mathsf{coreTree} : \mathsf{Tree} \to \mathsf{Tree}$ that strips cas-leaf and rec-leaf code wrappers, acting as a normal form for the equational theory generated by the computation axioms.

Definition 28. coreTree *is defined by mutual recursion:*

- $\mathsf{coreTree}\ \mathsf{lf} = \mathsf{lf}$
- $\mathsf{coreTree}\ (\mathsf{nd}\ \mathit{tag}\ \mathit{payload}) = \mathsf{coreDispatch}\ \mathit{tag}\ \mathit{payload}$

where coreDispatch *dispatches on the tag (with exhaustive, non-overlapping pattern matching for* `--exact-split`*):*

- *If tag* $=$ tagCase*: inspect the payload. If the payload is* $\mathsf{nd}\ (\mathsf{nd}\ \mathsf{lf}\ \mathsf{lf})\ (\mathsf{nd}\ c_U\ c_V)$ *(i.e., the scrutinee code is* $\mathsf{codeTerm}\ \mathsf{leaf} = \mathsf{nd}\ \mathsf{lf}\ \mathsf{lf}$*), return* $\mathsf{coreTree}\ c_U$*. Otherwise return the code unchanged.*
- *If tag* $=$ tagRec*: analogously, if the scrutinee code is* $\mathsf{nd}\ \mathsf{lf}\ \mathsf{lf}$*, return* $\mathsf{coreTree}\ c_Z$*. Otherwise unchanged.*
- *All other tags: return* nd *tag payload unchanged.*

The crucial property: coreTree computes the same value on both sides of any equation that thS can produce.

9.2 The CoreInv Predicate

Definition 29.

$$\mathsf{CoreInv}\ c = \begin{cases} \mathsf{Unit} & \text{if } c = \mathsf{lf} \\ \mathsf{Eq}\ (\mathsf{coreTree}\ L)\ (\mathsf{coreTree}\ R) & \text{if } c = \mathsf{nd}\ L\ R \end{cases}$$

Lemma 30 (CoreInv Preserved by Swap)**.** *If* $\mathsf{CoreInv}\ c$ *then* $\mathsf{CoreInv}\ (\mathsf{swapCode}\ c)$*.*

Proof. $\mathsf{swapCode}\ (\mathsf{nd}\ L\ R) = \mathsf{nd}\ R\ L$. Apply eqSym. □

Lemma 31 (CoreInv Preserved by TransCode)**.** *If* $\mathsf{CoreInv}\ e_1$ *and* $\mathsf{CoreInv}\ e_2$ *then* $\mathsf{CoreInv}\ (\mathsf{transCode}\ e_1\ e_2)$*.*

Proof. When $e_1 = \mathsf{nd}\ L_1\ R_1$, $e_2 = \mathsf{nd}\ L_2\ R_2$, and the middle matches ($\mathsf{eqTree}\ R_1\ L_2 = \mathsf{lf}$, so $R_1 = L_2$ by eqTree-sound), the result is $\mathsf{nd}\ L_1\ R_2$. The proof chains:

$$\mathsf{coreTree}\ L_1 = \mathsf{coreTree}\ R_1 = \mathsf{coreTree}\ L_2 = \mathsf{coreTree}\ R_2$$

where the middle step uses eqCong coreTree applied to $R_1 = L_2$. When the middle does not match, transCode returns defaultCode, which trivially satisfies CoreInv (both sides are $\mathsf{nd}\ \mathsf{lf}\ \mathsf{lf}$). □

Lemma 32 (CoreInv for Computation Axioms)**.**

(i) $\mathsf{CoreInv}\ (\mathsf{casLeafCode}\ u\ v) = \mathsf{Eq}\ (\mathsf{coreTree}\ u)\ (\mathsf{coreTree}\ u) = \mathsf{refl}$.

(ii) $\mathsf{CoreInv}\ (\mathsf{recLeafCode}\ z\ s) = \mathsf{Eq}\ (\mathsf{coreTree}\ z)\ (\mathsf{coreTree}\ z) = \mathsf{refl}$.

Proof. For (i): $\mathsf{casLeafCode}\ u\ v = \mathsf{nd}\ (\mathsf{nd}\ \mathsf{tagCase}\ (\mathsf{nd}\ (\mathsf{nd}\ \mathsf{lf}\ \mathsf{lf})\ (\mathsf{nd}\ u\ v)))\ u$. The left side's code has tag tagCase with scrutinee $\mathsf{nd}\ \mathsf{lf}\ \mathsf{lf} = \mathsf{codeTerm}\ \mathsf{leaf}$. By definition, coreTree strips this to $\mathsf{coreTree}\ u$, which equals $\mathsf{coreTree}\ u$ (the right side). This is definitional in Agda (proved by `refl`). Part (ii) is analogous. □

9.3 The Main Invariant

Theorem 33 (CoreInv for All Trees)**.** *For all* y : Tree*:*

$$\mathsf{CoreInv}\ (\mathsf{thS}\ y)$$

Proof. By structural induction on y, following the case structure of thS:

- Base and default branches: thS returns defaultCode or a reflexivity code. Both sides equal, so CoreInv holds by refl.
- Reflexivity: thS returns nd (codeReify p) (codeReify p). refl.
- Symmetry: by the inductive hypothesis and swap preservation.
- Cas-leaf / rec-leaf: by computation (definitional refl).
- Transitivity: by Lemma 31 with both inductive hypotheses.

Note: this theorem requires *no* ValidProof hypothesis. The invariant holds for *all* trees because every branch of thS preserves it structurally. □

9.4 Generalized Range Exclusion

Theorem 34 (Core-Unprovable)**.** *For all* L, R : Tree*, if* coreTree $L \neq$ coreTree R*, then for all* y : Tree*:*

$$\mathsf{thS}\ y \neq \mathsf{nd}\ L\ R$$

Proof. If thS y = nd L R, then by Theorem 33, coreTree L = coreTree R, contradicting the hypothesis. □

This is a powerful metatheorem: it gives an effective, decidable criterion for unprovability. Given any equation code nd L R, compute coreTree L and coreTree R; if they differ, the equation is *never* produced by thS, regardless of input.

9.5 Gödel I: True-but-Unprovable

Definition 35. *The* niter-leaf equation*:* niter If If (pair v_1 v_0) = leaf*.*

Theorem 36 (Niter-Leaf Equation is True)**.** eval (niter If If (pair v_1 v_0)) = eval leaf = If*.*

Proof. niter with clock If returns the state immediately: niter If st $s \to st$. Since st = If, both sides evaluate to If. (Proved by refl.) □

Theorem 37 (Niter-Leaf Equation is Unprovable)**.** *For all* y : Tree*:*

$$\mathsf{thS}\ y \neq \mathsf{nd}\ (\mathsf{codeTerm}\ (\mathsf{niter}\ \mathsf{If}\ \mathsf{If}\ (\mathsf{pair}\ v_1\ v_0)))\ (\mathsf{codeTerm}\ \mathsf{leaf})$$

Proof. By Theorem 34. The left side's code has tag tagNiter; the right side's code has tag tagLeaf = If. Neither is tagCase or tagRec, so coreTree returns both unchanged. Tag discrimination: tagNiter = nd If (nd If (nd If (nd If (nd If If)))) $\neq$ If = tagLeaf. □

This is Gödel's First Incompleteness Theorem in a strong form: a specific true equation lies outside the range of thS for *all* inputs, not just valid proof trees.

10 Meta-Level Consistency

Theorem 38 (Meta-Level Consistency)**.** *For all* y : Tree*:*

$$\mathsf{thS}\ y \neq \mathsf{nd}\ (\mathsf{codeTerm}\ \mathsf{leaf})\ (\mathsf{codeTerm}\ (\mathsf{pair}\ \mathsf{leaf}\ \mathsf{leaf}))$$

Proof. By Theorem 34. The left code codeTerm leaf = nd lf lf has coreTree value nd lf lf (tag lf = tagLeaf). The right code codeTerm (pair leaf leaf) has tag tagPair = nd lf (nd lf lf) ≠ lf. □

This establishes the consistency of thS as a metatheorem: the false equation "leaf = pair leaf leaf" is never produced by thS, regardless of input. This is strictly stronger than the soundness-based consistency proof (which requires ValidProof) because it holds for *all* trees.

11 Gödel II: The Consistency Sentence is Unprovable

11.1 The Consistency Sentence

Recall that godelSentence lf evaluates to falseT (Theorem 8.3), and the equation

$$\mathsf{godelSentence}\ \mathsf{lf} = \mathsf{reifyFalse}$$

is true (both sides evaluate to falseT). This equation is a consistency-type statement: it says that thS, applied to a specific self-referential input (the code of the Gödel schema), does not produce lf (the degenerate value).

11.2 Unprovability of the Consistency Sentence

Theorem 39 (Gödel II, Rose-Style)**.** *For all* y : Tree*:*

$$\mathsf{thS}\ y \neq \mathsf{nd}\ (\mathsf{codeTerm}\ (\mathsf{godelSentence}\ \mathsf{lf}))\ (\mathsf{codeTerm}\ \mathsf{reifyFalse})$$

Proof. By Theorem 34. The key structural observation:

- godelSentence lf is a cas-headed term (because matchSub always wraps its scrutinee in cas, and diag preserves the outermost constructor). Therefore codeTerm (godelSentence lf) has tag tagCase.
- The scrutinee of this cas (after diagonalization) is rec (reify c) ..., a rec-headed term. Its code has tag tagRec ≠ lf, so coreTree does *not* strip the cas-leaf wrapper. The tag remains tagCase.
- reifyFalse = pair leaf leaf has code with tag tagPair. Since this is neither tagCase nor tagRec, coreTree returns it unchanged.
- Tag discrimination: tagCase = nd lf (nd lf (nd lf lf)) ≠ nd lf (nd lf lf) = tagPair.

□

Remark 40 (The CoreTree Technique vs. Classical Approaches)**.** *The classical proof of Gödel II via the Löb/Hilbert–Bernays argument requires derivability conditions D1–D3 and an internal provability predicate. Our proof is purely* structural*: the* coreTree *quotient gives an effective range characterization of* thS *that makes both incompleteness theorems immediate consequences of tag discrimination. The tradeoff is that our Gödel II proves unprovability of a specific self-referential consistency sentence, rather than the abstract "if Con is provable then ⊥" form.*

12 What Remains

12.1 Stronger Gödel II

The current Gödel II uses godelSentence lf, whose consistency content is that thS at one self-referential input does not produce lf. A stronger version would use *target* = falseEqCode (the code of leaf = pair leaf leaf), making the consistency content explicit: "thS at a self-referential input does not prove the false equation." The proof is structurally identical (same tag discrimination) but involves larger Agda normalizations.

12.2 Classical Gödel II via D1–D3

For the full Löb-style argument, one would need:

- An internal provability predicate (existential quantification over proof trees), requiring a formula layer beyond the equational setting.
- D2 (distributing provability over implication) and D3 (provability of provability), which in the equational setting correspond to internal composition of proof trees via transCode.
- The Löb fixed-point argument composing these conditions.

A separate formalization (GuardBinaryGodel2.agda, 234 lines) carries this out in a predicate-logic setting over the same binary tree arithmetic.

12.3 Additional Computation Axioms

The current thS has cas-leaf and rec-leaf. For a more complete system:

- **cas-pair**: cas (pair (reify a) (reify b)) leaf v = reify ($\mathsf{eval}_{\rho[a,b]}\ v$).
- **rec-pair**: analogous, with 4-variable substitution on the RHS.
- **niter-leaf / niter-pair**: iteration base and step.

Adding these would require extending coreTree to handle the new axiom patterns, but the quotient technique generalizes: each new axiom $f(\ldots) = g(\ldots)$ contributes a new stripping rule to coreTree, and the CoreInv preservation proofs follow the same structure.

13 Design Principles

- **ASCII only**: No Unicode in Agda files.
- **Semi-explicit style**: Implicit arguments for types and indices, explicit for computational content.
- **No with-abstraction**: All case analysis via eqSubst, eqCong, eqTrans, or helper functions.
- **–safe –without-K –exact-split**: Maximum safety guarantees.
- **Guard-style, not Rose-style**: The fixed-point construction follows Guard's "diagonalize the code" approach rather than Rose's universal internal interpreter approach, avoiding the index-shifting obstruction that blocked earlier attempts at internalizing codedSubst.

- **Validity-restricted soundness**: Semantic soundness (TrueEqCode) is proved only for valid proof trees (ValidProof), not for all trees. This resolves the vacuous-truth compositionality obstruction (Remark 15).

14 Statistics

The Rose formalization comprises 13 Agda files:

- 10 core infrastructure files (Base through Internal)
- 3 higher-level files (DiagCode, FixedPoint, ThInt, Godel)
- 0 postulates, 0 holes, `--safe --without-K --exact-split`
- 36+ unit tests (all verified by `refl`)

Key theorems proved:

1. Coded substitution correctness (codedSubst-correct)
2. Diagonal correctness (diagCode-correct)
3. Fixed-point identity (diag-code)
4. Internal tree equality correctness (eqTree-check-main)
5. Internal theorem enumerator correctness (thTerm-correct)
6. Decode round-trip (decodeTerm n (codeTerm t) = just t)
7. Full soundness (ValidProof $\to$ ThSResult)
8. D_{thS} correctness (metalevel and internal)
9. Gödel sentence evaluation and fixed-point
10. Gödel I: true-but-unprovable equation (Theorem 37)
11. Meta-level consistency (Theorem 38)
12. Gödel II: consistency sentence unprovable (Theorem 39)